

ExpenseEase

Expense Tracker Mobile App
Built with React Native (Expo)

Project Details

Cross Platform Development (CPD) - Tiny Project
UG-7 Semester | BCA Honors | Division F
Parul University, Vadodara

Sr.	Name	Enrollment No.
1	Pravinkumar Choudhary	2305101270266
2	Anurag Kumar	2305101270020
3	Anurag Pareekh	2305101270021

GitHub: <https://github.com/speedprav/ExpenseEase.git>

1. Introduction

ExpenseEase is a cross-platform mobile application that helps users record and understand their daily spending. Users can add expenses with a title, amount and category, see the running total, and view a category-wise summary. The app is built with React Native using Expo, so one JavaScript codebase runs on both Android and iOS. All data is stored on the device, so the app works fully offline.

2. Problem Statement

Students and young professionals often lose track of small daily expenses, which makes it hard to see where their money goes. Paper notes and memory are unreliable, and many finance apps are complex or need an account. ExpenseEase provides a fast, simple and account-free way to log expenses and see the spending pattern at a glance.

3. Objectives

- Build a working cross-platform app from a single React Native (Expo) codebase.
- Implement full CRUD (add, view, edit, delete) for expenses.
- Persist data locally so it survives app restarts.
- Apply component-based design, navigation, state management and clean project structure.

4. Key Features

- **Add expense** with title, amount and category (Food, Travel, Shopping, Bills, Health, Other).
- **Edit or delete** any expense; deletion asks for confirmation.
- **Total spent** card that updates instantly.
- **Summary tab** with per-category totals and percentage bars.
- **Input validation** for empty titles and invalid amounts.
- **Offline storage** using AsyncStorage.

5. Technology Stack

Area	Technology
Framework	React Native with Expo (SDK 57), React 19
Language	JavaScript (ES6+)
Navigation	React Navigation - bottom tabs and native stack
State management	React Context API with hooks (useState, useEffect, useMemo)
Local storage	@react-native-async-storage/async-storage
Version control	Git and GitHub
Testing	Expo Go on physical device / Android emulator

6. System Design

The app follows a simple layered structure. Screens display data and collect input, a Context provider holds the expense list and all operations, and AsyncStorage keeps the data on the device.

Layer	Responsibility
Screens	HomeScreen (list + total), AddExpenseScreen (add/edit form), SummaryScreen (category breakdown)
Components	ExpenseItem - reusable card for one expense
Context	ExpenseContext - expenses array, add/update/delete functions, total, auto-save
Navigation	Bottom tabs (Expenses, Summary) inside a stack that opens the Expense form
Storage	AsyncStorage key @expenseease/expenses stores the list as JSON

Expense data model

```
{
  id: "17275900000000",
  title: "Lunch",
  amount: 120,
  category: "Food",
  date: "2026-09-29T08:00:00.000Z"
}
```

7. Project Structure

```
ExpenseEase/
|-- App.js                Root: providers + navigator
|-- index.js              Expo entry point
|-- app.json              Expo configuration
|-- package.json
|-- assets/               Icons and splash images
`-- src/
    |-- components/ExpenseItem.js
    |-- constants/         theme.js, categories.js
    |-- context/ExpenseContext.js
    |-- navigation/AppNavigator.js
    |-- screens/           HomeScreen.js, AddExpenseScreen.js,
    |                      SummaryScreen.js
    `-- utils/format.js
```

8. Screens and Workflow

Screen	What it does
Expenses (Home)	Shows the total-spent card and a list of expenses. The + button opens the form; tapping an item opens it for editing; Delete removes it after confirmation.
Expense form	Fields for title, amount and category chips. Validates input, then saves and returns to the list.
Summary	Lists each category with its amount, share of total, and a progress bar, sorted from highest to lowest.

9. Screenshots

Expenses screen	Add expense screen	Summary screen
-----------------	--------------------	----------------

10. How to Run

Requirements: Node.js 18 or newer, and the Expo Go app on a phone (or an Android emulator).

```
git clone https://github.com/speedprav/ExpenseEase.git
cd ExpenseEase
npm install
npx expo start
```

Scan the QR code with Expo Go (Android) or the Camera app (iOS). Press **a** in the terminal to open an Android emulator.

11. Challenges and Learnings

- Managing shared state cleanly with Context instead of passing props through many screens.
- Saving to AsyncStorage only after the initial load, so saved data is never overwritten by an empty list.
- Combining tab and stack navigation in one app.
- Structuring a React Native project into screens, components, context and utilities.

12. Future Scope

- Date filters and monthly budgets with alerts.
- Charts (pie/bar) for spending trends.
- Export to CSV or PDF.
- Cloud sync and login with Firebase.

13. Conclusion

ExpenseEase meets its goal of providing a clean, offline expense tracker from one React Native (Expo) codebase. The project gave the team hands-on experience with cross-platform development, navigation, state management and local persistence, and forms a base that can grow into a full personal-finance app.

14. Repository

GitHub	https://github.com/speedprav/ExpenseEase.git
Framework	React Native (Expo)